

● Analyse du cahier de charge et état d'avancement

Based on my analysis of the README.md file and the TODO comments in the codebase, here's a comprehensive overview of what has been implemented and what remains to be done for the marketplace-mvp project.

✅ Fonctionnalités déjà livrées (Phase 0 + socle Phase 1)

Selon le README, les fonctionnalités suivantes sont déjà implémentées :

Core Functionality

- Authentification complète : Inscription (acheteur/vendeur), connexion, JWT access (15 min) + refresh token rotatif et révocable, logout, consentements légaux obligatoires
- Rôles & sécurité : BUYER, SELLER, ADMIN avec Spring Security, RBAC, isolation stricte des vendeurs, mots de passe BCrypt
- Catalogue : Catégories (CRUD admin), produits multivendeur (prix en unités mineures, devise XOF), recherche, filtres, tri, pagination
- Boutiques : Profil boutique par vendeur avec nom, slogan, description, coordonnées, visuels, slug unique
- Panier : Multivendeur, quantités bornées par le stock réel, fusion des lignes
- Commandes : Checkout transactionnel avec décrémentation de stock sous verrou pessimiste, historiques complets des statuts, snapshot produit
- Documents légaux : Entités versionnées (DRAFT/PUBLISHED/ARCHIVED), pages publiques, API admin

- Audit : Journalisation des inscriptions, connexions, commandes, changements de statut, gestion boutique
- Qualité : Flyway pour migrations, tests unitaires, gestion centralisée des exceptions, Bean Validation

Stack Technique

- Backend : Java 21, Spring Boot 3.3, Spring Security + JWT, Spring Data JPA/Hibernate
- Frontend : React 18, Vite, TypeScript strict, Tailwind CSS + DaisyUI
- Base de données : PostgreSQL 16
- DevOps : Docker, Docker Compose

🚧 Fonctionnalités restant à implémenter

1. Paiement (Major item - marked as "Non implémenté" in README)

- Tâches spécifiques :
 - Remplacer l'implémentation actuelle de paiement physique par une vraie intégration PaymentProvider
 - Créer l'endpoint de webhook pour la confirmation asynchrone du paiement
 - Brancher un provider de paiement (TMoney, Flooz, carte, PayPal) derrière l'entité Payment
- Localisation du code :
 - marketplace/backend/src/main/java/com/marketplace/order/OrderService.java :
TODO PAYMENT

- marketplace/backend/src/main/java/com/marketplace/entity/Payment.java : Deux TODOs (intégration + webhook)

2. Commissions

- Tâches spécifiques :
 - Créer l'entité Commission
 - Calculer et créer les lignes de commission lors du checkout selon un taux configurable par vendeur/catégorie
- Localisation du code :
 - marketplace/backend/src/main/java/com/marketplace/order/OrderService.java : TODO COMMISSION

3. Livraison

- Tâches spécifiques :
 - Créer les entités Delivery, DeliveryZone, DeliveryFee
 - Intégrer une carte Leaflet/OSM pour la gestion des zones de livraison (focus sur région Maritime)
- Mentionné dans : Feuille de route (section 10 du README)

4. Live Shopping (Phase 2)

- Tâches spécifiques :
 - Créer un module live/ séparé
 - Implémenter WebSocket (STOMP) pour les sessions en direct

- Créer l'entité LiveSession
- Note : Le champ live_session_id est déjà présent dans la table des commandes
- Mentionné dans : Feuille de route (section 10 du README)
- Important : Le streaming vidéo ne doit pas transiter par le backend Spring Boot - prévoir un fournisseur spécialisé

5. Réseau social produit

- Tâches spécifiques :
 - Commentaires sur les produits
 - Réactions (likes, etc.)
 - système d'avis/notions
 - fonctionnalité de favoris/wishlist
- Mentionné dans : Feuille de route (section 10 du README)

6. Notifications

- Tâches spécifiques :
 - Créer l'entité Notification
 - Implémenter une abstraction multi-canaux (interne, email, push, SMS)
- Mentionné dans : Feuille de route (section 10 du README)

7. Internationalisation (i18n)

- Tâches spécifiques :
 - Mettre en place une structure de clés côté frontend

- Configurer les messages côté backend
- Français comme langue par défaut
- Mentionné dans : Feuille de route (section 10 du README)

8. Recherche avancée

- Tâches spécifiques :
 - Évaluer et intégrer une solution de recherche comme Meilisearch ou Elasticsearch
 - Créer l'abstraction nécessaire pour basculer facilement entre les implémentations
- Mentionné dans : Feuille de route (section 10 du README)

9. Amélioration du stockage d'images

- Tâches spécifiques :
 - Remplacer le stockage JSON actuel des URLs d'images par une entité dédiée `ProductImage`
 - Intégrer un `StorageProvider` pour le stockage cloud (Cloudinary mentionné dans les variables d'environnement)
- Localisation du code :
 - `marketplace/backend/src/main/java/com/marketplace/entity/Product.java` : TODO concernant les URLs d'images

 Résumé des TODOs dans le code

J'ai trouvé 5 TODOs spécifiques dans le code backend Java :

1. OrderService.java :

- TODO PAYMENT : appeler PaymentProvider.initiatePayment(order)
- TODO COMMISSION : créer les lignes Commission par SellerOrder

2. Payment.java :

- TODO PAYMENT : remplacer par une vraie intégration PaymentProvider
- TODO PAYMENT WEBHOOK : endpoint de confirmation asynchrone

3. Product.java :

- TODO : remplacer les URLs d'images sérialisées en JSON par une entité ProductImage + StorageProvider

 Prochaines étapes recommandées

1. Priorité haute : Implémenter le paiement (car marqué comme "Non implémenté" dans les fonctionnalités livrées)
2. Priorité moyenne : Implémenter les commissions (dépend étroitement du paiement)
3. Priorité moyenne : Améliorer le stockage d'images (améliore la maintenabilité)
4. Priorité basse à moyenne : Commencer à travailler sur les fonctionnalités de phase 2 (live shopping, réseaux sociaux, etc.)

Le projet possède une excellente base architecturale avec de nombreux éléments préparés (entités, statuts, commentaires TODO) qui faciliteront l'implémentation de ces fonctionnalités restantes.

